Dynamic Contextual Embedding Engine (DCEE)

A Runtime Embedding Bridge for DCF, FSCS, and Field-Weighted Semantic Systems

Kevin T.N
Vietnam
keedavinci@gmail.com

March 18, 2026

Abstract

This paper defines the Dynamic Contextual Embedding Engine (DCEE), a runtime embedding specification intended to bridge theoretical semantic architectures and executable language systems. The core problem addressed is that many current embedding implementations remain static, token-local, and semantically opaque, even when the surrounding theoretical framework already assumes dynamic context, semantic field resonance, profile activation, and node-based meaning formation. In such cases, the conceptual architecture may be rich, but the runtime embedding layer still collapses input into fixed or weakly contextual vectors.

DCEE is proposed as the missing bridge. Instead of treating embedding as a function of input token alone, DCEE defines embedding as a function of input, active context plane, semantic memory, and field state. Its runtime form is therefore

$$E(x \mid C, \mathcal{M}, \mathcal{F}),$$

where x is the input unit or sequence, C is the active context plane, $\mathcal{M}$ is semantic memory, and $\mathcal{F}$ is the current semantic field state. The engine proceeds through six stages: symbol mapping, node resolution, context plane construction, resonance computation, profile activation, field projection, and final vector synthesis. The resulting embedding is dynamic by construction: the same lexical item may yield different coordinates under different contexts.

The present document does not aim to define a complete production implementation. Rather, it formalizes the runtime layer that must exist if a dynamic semantic theory is to become an operational embedding system. In that sense, DCEE should be read as an implementation bridge paper: the missing computational specification that connects symbol-first semantic learning, dynamic context fields, and field-based semantic weighting to actual runtime embeddings usable by downstream models.

Contents

1 Introduction 3

2 Motivation 3

3 Design Position 4

4 Core Definitions 4

5 Formal Embedding Objective 5

6 Runtime Pipeline 5

7 Step 1: Symbol Mapping 6

8 Step 2: Semantic Node Resolution 6

9 Step 3: Context Plane Construction 7

10 Step 4: Resonance Computation 7

11 Step 5: Profile Activation 7

12 Step 6: Field-Weight Projection 8

13 Step 7: Final Embedding Synthesis 8

14 Dynamic Property 9

15 Memory Integration 9

16 Field Integration 10

17 Minimal Computational Complexity 10

18 Comparison with Traditional Embeddings 10

19 Why This Bridge Is Necessary 10

20 Implementation Requirements 11

21 Worked Conceptual Example 11

22 Limitations 12

23 Conclusion 12

1. Introduction

Many systems now claim to have "semantic" representations while still relying on runtime embeddings that are either static or only weakly contextual. A common pattern is that the theoretical framework is rich, but the actual embedding step remains shallow. Input text is hashed, bucketed, averaged, or mapped to a token lookup table, producing vectors that may be numerically convenient but semantically thin. In such systems, two different utterances containing similar character sets may yield nearly indistinguishable vectors despite having different meanings, relations, or intentions.

The core issue is not always data scarcity. In many cases, the system already has meaningful internal resources:

- symbol memory,
- token or node memory,
- contextual histories,
- field structures,
- semantic profiles,
- graph relations.

What is missing is the runtime bridge that actually uses those structures when generating an embedding.

This paper proposes that bridge under the name *Dynamic Contextual Embedding Engine* (DCEE). The goal is to replace the idea of embedding as a fixed mapping from input to vector with a dynamic mapping from input and active semantic state to a contextual coordinate. In short, DCEE answers the question: if the system already assumes context-sensitive meaning, semantic fields, and node-based semantics, what should the actual embedding function look like at runtime?

Core claim. If meaning is context-dependent, field-conditioned, and node-mediated, then the runtime embedding function must also be context-dependent, field-conditioned, and node-mediated.

2. Motivation

The motivation for DCEE comes from the mismatch between theory and implementation in many emerging semantic systems. Three layers are often present:

1. a semantic or ontological theory that says meaning is dynamic,
2. a memory or substrate architecture that stores semantic history,
3. an implementation layer that still uses static or semi-static embeddings.

This mismatch becomes visible when the system claims that:

- the same word should mean different things in different contexts,
- semantic units have profiles,
- fields affect interpretation,
- node lineage matters,

but then computes embeddings using only token identity, raw symbol hashing, or frequency-only lookup.

DCEE is therefore motivated by a simple requirement: the runtime embedding must reflect the system's own semantics.

3. Design Position

DCEE should be understood as an *engine specification*, not as a complete model of language. It is not a replacement for all downstream architectures. It is also not a claim that one single formula solves semantic representation in every context. Rather, it is a disciplined runtime formulation for systems that already assume:

- symbol-first or node-based semantics,
- contextual meaning,
- dynamic profile activation,
- field weighting,
- semantic memory.

What DCEE is. A runtime embedding bridge from dynamic semantic theory to executable embedding vectors.
What DCEE is not. A full language model, a total semantic theory, or a claim that static embeddings are universally useless.

4. Core Definitions

Definition 4.1 (Input Unit)**.** *An* input unit x *is the textual or symbolic object to be embedded. It may be a symbol, token, word, phrase, or short sequence.*

Definition 4.2 (Semantic Node)**.** *A* semantic node *is a learned semantic unit in memory representing a stabilized concept, cluster, or lexical-semantic entity. Nodes are denoted by* $n \in \mathcal{N}$.

Definition 4.3 (Context Plane)**.** *A* context plane C *is the active semantic neighborhood constructed from surrounding nodes, discourse state, or ongoing internal state. It provides the runtime frame in which the input is interpreted.*

Definition 4.4 (Semantic Memory)**.** $\mathcal{M}$ *denotes the semantic memory store. It contains semantic nodes, profile sets, lineage data, context histories, and possibly lexical or phrase families.*

Definition 4.5 (Field State)**.** *$\mathcal{F}$ denotes the current semantic field state, usually represented by a finite set of fields and node-to-field weights.*

Definition 4.6 (Dynamic Embedding)**.** *A* dynamic embedding *is an embedding whose value changes as a function of active context, semantic memory, and field state, even when the lexical input remains constant.*

5. Formal Embedding Objective

The standard static embedding objective is:

$$E(x).$$

The DCEE objective is:

$$E(x \mid C, \mathcal{M}, \mathcal{F}).$$

This states that embedding is not a function of the input alone. It is a function of:

- what the input is,
- what context is currently active,
- what semantic memory the system already has,
- what field configuration the system is currently using.

Principle 5.1. *If two identical input tokens occur in different contexts, their embeddings should in general differ.*

6. Runtime Pipeline

The runtime pipeline is defined as:

$$x \to S \to N \to C \to R \to W \to V,$$

where:

- S: symbol representation,
- N: semantic node resolution,
- C: context plane construction,
- R: resonance computation,
- W: field-weight projection,
- V: final embedding vector.

Expanded:

$$\begin{aligned} x &\mapsto \text{normalized symbols or token fragments}, \\ \text{symbols} &\mapsto \text{candidate nodes}, \end{aligned}$$

$$\begin{aligned}
\text{candidate nodes} &\mapsto \text{active context plane},\\
\text{active context plane} &\mapsto \text{node resonance},\\
\text{node resonance} &\mapsto \text{profile activation},\\
\text{profile activation} + \text{field weights} &\mapsto \text{contextual node coordinates},\\
\text{contextual node coordinates} &\mapsto V.
\end{aligned}$$

7. Step 1: Symbol Mapping

Input x is first normalized into a sequence of internal representational units:

$$S = (s_1, s_2, \ldots, s_k).$$

This stage does not yet decide final meaning. It simply ensures that the input is brought into the system's internal symbol space.

Depending on the larger architecture, this stage may include:

- character-level normalization,
- Unicode normalization,
- lookup into symbol memory,
- decomposition into smaller symbolic units.

8. Step 2: Semantic Node Resolution

The engine then attempts to resolve the input into one or more semantic nodes:

$$N(x, \mathcal{M}) = \{n_1, n_2, \ldots, n_m\}.$$

This may be:

- a direct lexical lookup,
- a cluster match,
- a node graph traversal,
- a fallback reconstruction from symbol structure.

Each node n_i is assumed to carry:

- one or more semantic profiles,
- a coordinate representation,
- field-weight information,
- lineage or context history.

9. Step 3: Context Plane Construction

Let the surrounding semantic neighborhood of the current input be:

$$C = \{n_{i-w}, \ldots, n_{i-1}, n_{i+1}, \ldots, n_{i+w}\},$$

where the neighborhood may be constructed from:

- neighboring tokens,
- previously activated nodes,
- discourse memory,
- active conversational state,
- user or agent-specific semantic history.

A simple context-plane vector is:

$$C_{\text{vec}} = \frac{1}{|C|} \sum_{n_j \in C} p_{n_j},$$

where p_{n_j} is the current coordinate of node n_j.

Remark 9.1. *This is only the simplest linear construction. Nonlinear context planes, weighted context planes, or graph-induced context manifolds may also be used.*

10. Step 4: Resonance Computation

For node n_i, resonance with the current context plane is defined by:

$$R(n_i, C) = \frac{p_{n_i} \cdot C_{\text{vec}}}{\|p_{n_i}\| \; \|C_{\text{vec}}\|}.$$

This is a cosine-like similarity form. More general forms may include:

- field-adjusted resonance,
- lineage-aware resonance,
- profile-specific resonance,
- nonlinear projection-based resonance.

A more general operator may be written:

$$R(n_i, C, \mathcal{F}, \mathcal{M}) = \text{Res}(n_i \mid C, \mathcal{F}, \mathcal{M}).$$

11. Step 5: Profile Activation

Each node may carry multiple semantic profiles:

$$\mathcal{P}_{n_i} = \{p_{i1}, p_{i2}, \ldots, p_{iq}\}.$$

These profiles correspond to different meaning modes or contextual realizations of the same node.

For each profile p_{ij}, define profile resonance:

$$R_{ij} = \mathrm{Res}(p_{ij} \mid C, \mathcal{F}, \mathcal{M}).$$

Activation weights are then:

$$a_{ij} = \frac{\exp(R_{ij}/T)}{\sum_{k=1}^{q} \exp(R_{ik}/T)},$$

where $T > 0$ is a temperature parameter.

The active contextual node coordinate becomes:

$$p_{n_i}(C) = \sum_{j=1}^{q} a_{ij} p_{ij}.$$

Principle 11.1. *A node should not be represented by one fixed coordinate if its semantic role changes across contexts. Instead, its runtime coordinate should be the activated mixture of its profiles.*

12. Step 6: Field-Weight Projection

Each node n_i maintains field weights:

$$W_{n_i} = [w_{i1}, w_{i2}, \ldots, w_{i6}], \qquad \sum_{k=1}^{6} w_{ik} = 1.$$

Let the six field basis vectors be:

$$\mathcal{F} = \{F_1, F_2, \ldots, F_6\}.$$

The field contribution of node n_i is:

$$F(n_i) = \sum_{k=1}^{6} w_{ik} F_k.$$

Field-adjusted node position:

$$p_i' = p_{n_i}(C) + \lambda\, F(n_i),$$

where $\lambda \geq 0$ controls how strongly field structure influences runtime embedding.

13. Step 7: Final Embedding Synthesis

Let $A \subseteq \mathcal{N}$ be the set of active nodes for the current input. Define node-level synthesis weights:

$$\alpha_i = \frac{\exp(R(n_i, C)/\tau)}{\sum_{n_j \in A} \exp(R(n_j, C)/\tau)},$$

where $\tau > 0$ is a synthesis temperature.

The final embedding vector is:

$$V = \sum_{n_i \in A} \alpha_i p'_i.$$

Therefore the runtime embedding is:

$$E(x \mid C, \mathcal{M}, \mathcal{F}) = V.$$

14. Dynamic Property

The most important consequence of this design is the dynamic property:

$$E(x \mid C_1, \mathcal{M}, \mathcal{F}) \neq E(x \mid C_2, \mathcal{M}, \mathcal{F}) \quad \text{in general.}$$

For example:

$$E(\text{bank} \mid \text{river context}) \neq E(\text{bank} \mid \text{finance context}).$$

Likewise:

$$E(\text{cá} \mid \text{food context}) \neq E(\text{cá} \mid \text{biology context}).$$

This is not a side effect. It is the central purpose of the engine.

15. Memory Integration

Semantic memory $\mathcal{M}$ is not an optional add-on. It is an explicit input to the engine. Memory may include:

- node definitions,
- node profiles,
- lineage relations,
- context histories,
- phrase families,
- user- or agent-specific semantic traces.

Thus the full operational form remains:

$$E(x \mid C, \mathcal{M}, \mathcal{F}).$$

The same input in two different memories may yield different embeddings:

$$E(x \mid C, \mathcal{M}_1, \mathcal{F}) \neq E(x \mid C, \mathcal{M}_2, \mathcal{F}).$$

This provides a route toward semantic personalization without collapsing all semantics into hidden parameter drift.

16. Field Integration

Field state $\mathcal{F}$ enters the engine in three ways:

1. as a modifier of resonance,
2. as a contributor to node position,
3. as a control over semantic emphasis and attractor structure.

This means that embedding is not merely contextual; it is also field-conditioned. A system with different internal field balances may interpret the same context differently.

17. Minimal Computational Complexity

Let:

- n: number of active nodes,
- p: average number of profiles per node,
- f: number of fields.

Then a minimal runtime cost can be approximated by:

$$O(n \cdot p + n \cdot f).$$

This is much smaller than unrestricted full-attention behavior over large token sets, especially if node activation is already sparsified.

18. Comparison with Traditional Embeddings

Aspect	**Traditional Embedding**	**DCEE**
Runtime form	$E(x)$	$E(x \mid C, \mathcal{M}, \mathcal{F})$
Vector type	static or weakly contextual	strongly dynamic
Semantic structure	implicit	explicit node/profile/field form
Context dependence	indirect	native
Field awareness	absent	native
Memory dependence	often external	native
Interpretability	low	higher

19. Why This Bridge Is Necessary

A semantic system may already possess:

- a node graph,

- field structures,
- context logic,
- dynamic profile theory,

yet still fail semantically at runtime if embedding remains static.

That is why DCEE is necessary. It is the operational bridge that forces runtime representation to obey the system's own theoretical commitments.

Bridge claim. Without a runtime engine such as DCEE, a context-dependent semantic theory may still collapse into a static embedding practice.

20. Implementation Requirements

A minimal DCEE implementation requires:

- semantic node memory,
- node profile store,
- context plane builder,
- resonance calculator,
- field-weight table,
- vector synthesis module.

A stronger implementation may further include:

- lineage-aware node activation,
- family-word retrieval hooks,
- personalized field resonance,
- dynamic memory updates after successful response generation.

21. Worked Conceptual Example

Suppose the input is the token $x =$ "cá". Let semantic memory resolve it to a node $n_{\text{cá}}$ with profiles:

$$\mathcal{P}_{n_{\text{cá}}} = \{p_{\text{animal}}, p_{\text{food}}, p_{\text{culture}}\}.$$

Now suppose the active context plane is generated from nodes:

$$C = \{\text{ăn}, \text{chiên}, \text{món}\}.$$

Then resonance values may satisfy:

$$R_{\text{food}} > R_{\text{animal}} > R_{\text{culture}}.$$

Profile activation weights become:

$$a_{\text{food}} \gg a_{\text{animal}}, a_{\text{culture}}.$$

So:

$$p_{n_{\text{cá}}}(C) \approx a_{\text{food}} p_{\text{food}} + a_{\text{animal}} p_{\text{animal}} + a_{\text{culture}} p_{\text{culture}}.$$

If the node also carries strong weights on a culinary field, then field projection shifts the coordinate further toward a food-oriented region. The final embedding is therefore a food-context embedding of *cá*, not a neutral or static lexical vector.

22. Limitations

DCEE is a runtime bridge specification, not a full semantic proof. Several open problems remain:

- how to best build nonlinear context planes,
- how to update semantic memory online,
- how to calibrate field influence without over-biasing,
- how to avoid profile explosion,
- how to compress deeply nested node families.

These are not objections to the engine; they are the next implementation questions.

23. Conclusion

This paper defined the Dynamic Contextual Embedding Engine as a runtime bridge from dynamic semantic theory to executable embedding computation. The key shift is that embedding should no longer be treated as a fixed mapping from input to vector. Instead, it should be treated as a contextual semantic coordinate synthesized from node resolution, context plane construction, profile activation, field projection, and semantic memory.

The resulting form,

$$E(x \mid C, \mathcal{M}, \mathcal{F}),$$

captures the minimal structure required for systems whose semantics are already assumed to be dynamic. DCEE therefore occupies a clear architectural role: it is the operational layer that makes dynamic semantic theory computable at runtime.

Appendix A: Compact Equation Summary

$$E(x \mid C, \mathcal{M}, \mathcal{F})$$

$$C_{\text{vec}} = \frac{1}{|C|} \sum_{n_j \in C} p_{n_j}$$

$$R(n_i, C) = \frac{p_{n_i} \cdot C_{\text{vec}}}{\|p_{n_i}\| \ \|C_{\text{vec}}\|}$$

$$a_{ij} = \frac{\exp(R_{ij}/T)}{\sum_k \exp(R_{ik}/T)}$$

$$p_{n_i}(C) = \sum_j a_{ij} p_{ij}$$

$$F(n_i) = \sum_{k=1}^{6} w_{ik} F_k$$

$$p'_i = p_{n_i}(C) + \lambda F(n_i)$$

$$\alpha_i = \frac{\exp(R(n_i, C)/\tau)}{\sum_{n_j \in A} \exp(R(n_j, C)/\tau)}$$

$$V = \sum_{n_i \in A} \alpha_i p'_i$$

$$E(x \mid C, \mathcal{M}, \mathcal{F}) = V$$

Appendix B: Minimal Module List

Module	**Role**
Symbol Mapper	Normalize input into internal symbol units
Node Resolver	Map input symbols/tokens to semantic nodes
Context Plane Builder	Construct active context plane from neighboring nodes
Resonance Calculator	Compute node/profile resonance against context
Profile Activator	Produce context-dependent profile weights
Field Projector	Add field-conditioned semantic displacement
Vector Synthesizer	Combine active node coordinates into final embedding